

cloudforger Developer Reference

A guide to the `src/cloudforger` package

Auto-generated from source

June 2026

Abstract

This document is the authoritative developer reference for the **cloudforger** Python package located at `src/cloudforger/` within the *point-process-tda* research project. It covers every public class and function, the end-to-end data pipeline (from process simulation to neural-network training), and a minimal worked example that a new contributor can run immediately. All content is derived directly from the source code; nothing is assumed or invented.

Contents

1	Package Overview	4
1.1	Purpose and Scope	4
1.2	Package Layout	4
1.3	Key External Dependencies	4
2	Core Data Structures	4
2.1	Region and Box	4
2.2	PointCloud	5
2.3	PointProcess	6
2.4	PersistenceDiagram	7
2.5	CorrelationFeatures	7
3	Point Processes	7
3.1	PoissonProcess	7
3.2	ThomasProcess	8
3.3	MaternHardCoreProcess (Type II)	8
4	Cloud Generation Pipeline	9
4.1	End-to-End Data Flow	9
4.2	Generating Clouds (scripts)	9
5	TDA Subsystem	10
5.1	Filtrations	10
5.1.1	Filtration (abstract)	10
5.1.2	RipsFiltration	10
5.2	Diagram Calibration	10
5.3	Vectorization	11
5.3.1	PersistenceImager	11
5.3.2	MultiChannelImager	11
6	Statistics Subsystem	12
6.1	CloudStatistic (abstract)	12
6.2	PairDistanceCDF	12
7	Neural Network Subsystem	12
7.1	Datasets	12
7.1.1	PointCloudDataset	13
7.1.2	PersistenceImageDataset	13
7.1.3	CorrelationFeatureDataset	13
7.2	Encoders	13
7.2.1	PointNetEncoder	14
7.2.2	PIEncoder	14
7.2.3	StatsEncoder	15
7.3	Task Heads	15
7.3.1	ClassificationHead	15
7.3.2	ParameterEstimator	15
7.4	Models	15
7.4.1	SingleModalModel	15
7.4.2	MultiModalModel	16

7.5	Training Utilities	17
7.5.1	<code>train_val_test_split()</code>	17
7.5.2	<code>train_one_epoch()</code> and <code>evaluate()</code>	17
7.6	<code>standardize_size()</code>	17
8	Public API Quick Reference	18
9	Usage Walkthrough	19
9.1	Step 1: Generate Clouds	19
9.2	Step 2: Compute Persistence Diagrams	20
9.3	Step 3: Calibrate and Vectorize	20
9.4	Step 4: Build Datasets and Splits	20
9.5	Step 5: Assemble and Train a Model	21
10	Known Ambiguities and Implementation Notes	21

1 Package Overview

1.1 Purpose and Scope

`cloudforger` is a research-grade Python library for generating spatial point clouds from classical stochastic point processes, computing topological and statistical descriptors of those clouds, and routing the resulting feature representations into PyTorch neural networks for classification or parameter estimation.

The package sits inside the broader *point-process-tda* project, which investigates whether topological data analysis (TDA) features can distinguish point-process types across varying ambient dimensions. `cloudforger` is the computational engine; the surrounding `scripts/` directory contains the experiment runners and processing scripts that orchestrate it.

1.2 Package Layout

Subpackage / file	Responsibility
<code>core/</code>	Shared data structures (<code>PointCloud</code> , <code>Region</code> , <code>PersistenceDiagram</code> , <code>CorrelationFe</code>)
<code>processes/</code>	Point-process implementations (Poisson, Thomas, Matérn II)
<code>stats/</code>	Empirical summary statistics (pair-distance CDF)
<code>tda/filtration/</code>	Persistent-homology computation (Rips filtration)
<code>tda/vectorizer/</code>	Diagram-to-image vectorization (persistence images)
<code>tda/calibration.py</code>	Automatic bound selection for vectorizers
<code>nn/data.py</code>	PyTorch <code>Dataset</code> wrappers
<code>nn/encoders/</code>	Modality-specific neural encoders
<code>nn/heads/</code>	Task heads (classification, parameter estimation)
<code>nn/models/</code>	Assembled model wrappers (<code>Single-</code> / <code>MultiModalModel</code>)
<code>nn/train.py</code>	Training and evaluation loop functions
<code>nn/splits.py</code>	Reproducible train/val/test splitting

1.3 Key External Dependencies

Package	Role
<code>numpy</code>	Array arithmetic and seeded random number generation
<code>scipy</code>	<code>cKDTree</code> for nearest-neighbour queries (Matérn process)
<code>ripser</code>	Efficient Vietoris–Rips persistent homology
<code>torch</code>	Tensor computation, neural networks, datasets
<code>pickle</code> / <code>yaml</code>	Dataset serialization and configuration loading

2 Core Data Structures

2.1 Region and Box

`src/cloudforger/core/region.py`

Region is an abstract base class representing a spatial domain. Every point process samples exclusively within a **Region**.

```
class Region(ABC):
    @property
    @abstractmethod
```

```

def dimension(self) -> int: ...

@property
@abstractmethod
def volume(self) -> float: ...

@abstractmethod
def contains(self, point: np.ndarray) -> np.ndarray: ...
# Returns boolean mask of shape (N,)

@abstractmethod
def sample_uniform(self, n: int,
                    rng: np.random.Generator) -> np.ndarray: ...
# Returns (n, dimension) array

@abstractmethod
def expanded(self, pad: float) -> "Region": ...
# Returns a new Region grown by 'pad' on every side

```

Box is the only concrete implementation. It represents an axis-aligned hyperrectangle $[l, h]^d$ defined by two 1-D numpy arrays `low` and `high`.

```

class Box(Region):
    def __init__(self, low: np.ndarray, high: np.ndarray):
        # low < high required element-wise; both must be 1-D
        ...
        # volume = prod(high - low)
        # contains uses >= low and <= high (closed box)
        # expanded(pad) returns Box(low - pad, high + pad)

```

Usage example:

```

import numpy as np
from cloudforger.core.region import Box

# Unit square in 2-D
region = Box(low=np.zeros(2), high=np.ones(2))
print(region.volume)      # 1.0
print(region.dimension) # 2

```

2.2 PointCloud

`src/cloudforger/core/cloud.py`

The central data structure of the package. It bundles raw point coordinates with provenance metadata.

Field	Type	Description
points	np.ndarray	Shape (N, D) — coordinates of N points in D dimensions
generator_name	str	Name of the process that produced this cloud
generator_params	dict	Process hyperparameters at sampling time
seed	int None	RNG seed for reproducibility
region	Region None	Sampling domain (required by some statistics)
n_points	int	(property) points.shape[0]
dimension	int	(property) points.shape[1]

PointCloud is a @dataclass; all fields are stored as-is (no copies made).

```
from cloudforgery.core.cloud import PointCloud
import numpy as np

pts = np.random.rand(200, 3)
cloud = PointCloud(
    points=pts,
    generator_name="example",
    generator_params={},
    seed=42,
)
print(cloud)
# PointCloud(n=200, d=3, generator='example', seed=42)
```

2.3 PointProcess

src/cloudforgery/core/base.py

Abstract base class that all point processes extend. Subclasses must implement `name`, `params`, and the internal sampling method `_sample_points`. The **public API** is the single method `sample()`.

```
class PointProcess(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...

    @property
    @abstractmethod
    def params(self) -> dict[str, Any]: ...

    @abstractmethod
    def _sample_points(self, n: int, region: Region,
                      rng: np.random.Generator) -> np.ndarray: ...

    def sample(self, n: int, region: Region,
               seed: int | None = None) -> PointCloud:
        rng = np.random.default_rng(seed)
        points = self._sample_points(n, region, rng)
        return PointCloud(points=points,
                          generator_name=self.name,
                          generator_params=self.params,
                          seed=seed,
                          region=region)
```

The argument `n` carries different semantics depending on the subclass — it may be the exact output count (Poisson without intensity) or an unused nominal target (Thomas / Matérn produce a random count regardless). See Section 3 for details.

2.4 PersistenceDiagram

`src/cloudforger/core/diagram.py`

Stores the output of a persistent-homology computation.

Field	Type	Description
<code>diagrams</code>	<code>dict[int, np.ndarray]</code>	Maps homology dimension k to an array of shape $(M_k, 2)$
<code>generator_name</code>	<code>str</code>	Provenance: process that produced the source cloud
<code>generator_params</code>	<code>dict</code>	Process parameters
<code>seed</code>	<code>int None</code>	RNG seed
<code>filtration_name</code>	<code>str</code>	Filtration type (e.g. "rips")
<code>filtration_params</code>	<code>dict</code>	Filtration parameters
<code>dimensions()</code>	<code>list[int]</code>	<i>(method)</i> Sorted list of stored homology dimensions
<code>finite_pairs(dim)</code>	<code>np.ndarray</code>	<i>(method)</i> Pairs with both birth <i>and</i> death finite

Death values of $+\infty$ (i.e. the feature never dies) are present in raw `ripser` output; `finite_pairs(dim)` removes them.

2.5 CorrelationFeatures

`src/cloudforger/core/features.py`

Wraps the output of a `CloudStatistic` computation.

Field	Type	Description
<code>features</code>	<code>dict[str, np.ndarray]</code>	Maps statistic name to its 1-D feature vector
<code>generator_name</code>	<code>str</code>	Source process name
<code>generator_params</code>	<code>dict</code>	Source process parameters
<code>seed</code>	<code>int None</code>	RNG seed
<code>statistic_params</code>	<code>dict</code>	Parameters of the computing statistic
<code>names()</code>	<code>list[str]</code>	<i>(method)</i> Sorted statistic names
<code>vector(names)</code>	<code>np.ndarray</code>	<i>(method)</i> Concatenation of vectors for the given names (all

3 Point Processes

3.1 PoissonProcess

`src/cloudforger/processes/poisson.py`

```
class PoissonProcess(PointProcess):
    def __init__(self, intensity: float | None = None):
        ...
```

Two modes depending on whether `intensity` is supplied:

- **Conditional mode** (`intensity=None`): Samples exactly n points uniformly at random in the region. This is the conditional Poisson process given count n .
- **Homogeneous mode** (`intensity= $\lambda > 0$`): Draws a random count $N \sim \text{Poisson}(\lambda \cdot |W|)$ and then places N points uniformly in the region W .

The parameter name in `params` is "intensity"; the process name is "poisson".

3.2 ThomasProcess

`src/cloudforger/processes/thomas.py`

The Thomas process (Neyman–Scott with Gaussian scatter kernel) generates clustered patterns.

```
class ThomasProcess(PointProcess):
    def __init__(self,
        parent_intensity: float, # lambda_p > 0
        mean_offspring: float, # mu > 0
        cluster_scale: float): # sigma > 0
        ...
```

Sampling algorithm (edge-corrected):

1. Expand the sampling region by 4σ on every side to create an enlarged guard region.
2. Sample $N_p \sim \text{Poisson}(\lambda_p \cdot |W_{\text{expanded}}|)$ parent centres uniformly in the enlarged region.
3. For each parent i , draw offspring count $K_i \sim \text{Poisson}(\mu)$.
4. Scatter offspring around each parent: $\mathbf{x}_{ij} = \mathbf{c}_i + \boldsymbol{\epsilon}_{ij}$, where $\boldsymbol{\epsilon}_{ij} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$.
5. Retain only offspring that fall inside the *original* region.

The guard region ensures that offspring landing inside the original region from parents located just outside it are correctly simulated. The process name is "thomas".

3.3 MaternHardCoreProcess (Type II)

`src/cloudforger/processes/matern.py`

Produces repulsive (inhibitory) patterns via dependent thinning.

```
class MaternHardCoreProcess(PointProcess):
    def __init__(self,
        parent_intensity: float, # lambda_p > 0
        hardcore_radius: float): # r > 0
        ...
```

Sampling algorithm:

1. Sample a parent Poisson process with intensity λ_p in the region.
2. Assign independent uniform marks $u_i \sim \text{Uniform}(0, 1)$ to every parent.
3. Build a k -d tree over the parent locations.
4. Retain point i if and only if *no* neighbour within distance r has a strictly smaller mark than u_i .

This is the Type II construction; retained points are guaranteed to be mutually at distance $> r$.

The process name is "matern_ii".

4 Cloud Generation Pipeline

4.1 End-to-End Data Flow

Figure 1 shows the complete journey from process parameters to neural-network inputs, as implemented in the `scripts/` directory using the `cloudforg` primitives.

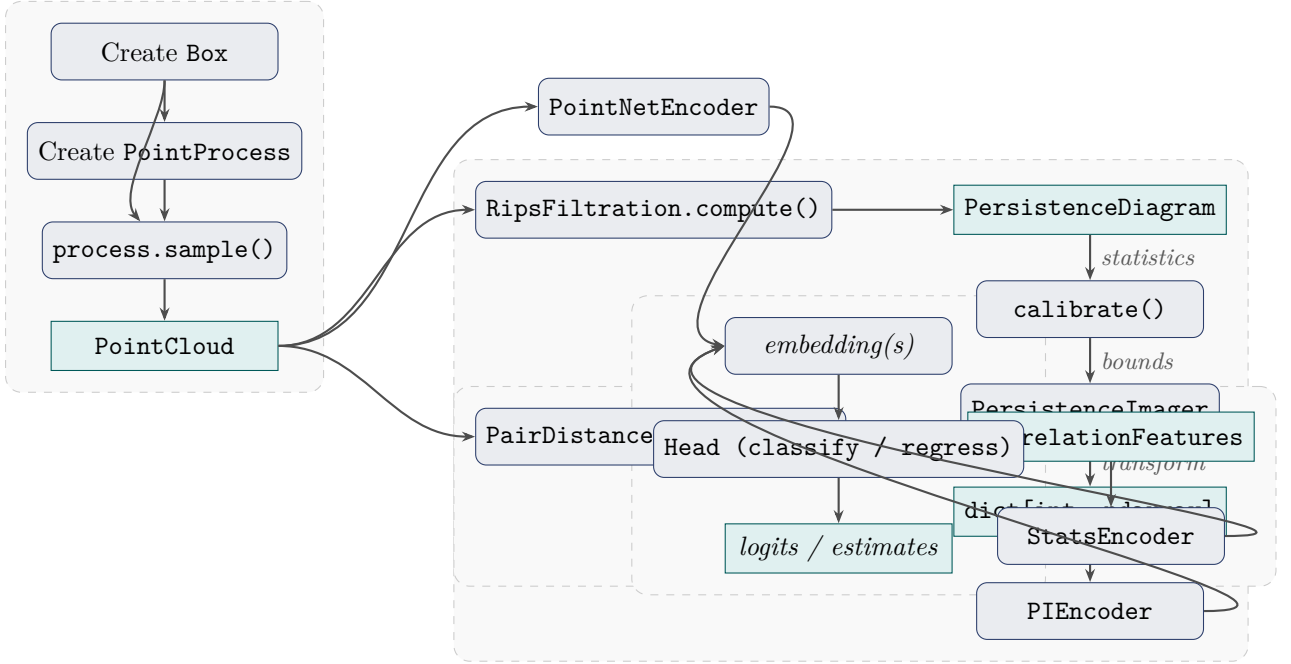


Figure 1: End-to-end pipeline from process parameters to model predictions. The raw-point-cloud branch (`PointNetEncoder`) bypasses both TDA and statistics and operates directly on the $(N \times D)$ array. All three branches produce an embedding that feeds the same classification or regression head.

4.2 Generating Clouds (scripts)

In practice the pipeline is driven by three pre-processing scripts:

1. `scripts/processing/classify/generate_clouds.py`
Calibrates process parameters so that the expected output count ≈ 500 for each process and ambient dimension d . The calibration uses binary search on `parent_intensity`. Serialises a list of `PointCloud` objects to `data/{d}d/clouds.pkl`.
2. `scripts/processing/classify/compute_diagrams.py`
Reads `clouds.pkl`, runs `RipsFiltration(maxdim=1)`, writes `data/{d}d/diagrams.pkl`.
3. `scripts/processing/classify/vectorize_diagrams.py`
Reads `diagrams.pkl`, calls `calibrate()` to pick 99th-percentile bounds, constructs one `PersistenceImager` per homology dimension, applies `MultiChannelImager`, writes `data/{d}d/images.pkl`.
4. `scripts/processing/classify/compute_pairwise.py`
Reads `clouds.pkl`, runs `PairDistanceCDF`, writes `data/{d}d/features.pkl`.

5 TDA Subsystem

5.1 Filtrations

5.1.1 Filtration (abstract)

src/cloudforger/tda/filtration/base.py

```
class Filtration(ABC):
    def compute(self, cloud: PointCloud) -> PersistenceDiagram:
        # calls _compute_diagrams then wraps result with metadata
        ...

    @abstractmethod
    def _compute_diagrams(self, cloud: PointCloud) -> dict[int, np.
        ndarray]: ...
```

5.1.2 RipsFiltration

src/cloudforger/tda/filtration/rips.py

```
class RipsFiltration(Filtration):
    def __init__(self, maxdim: int = 1, thresh: float | None = None):
        ...
        # Delegates to ripser(cloud.points, maxdim=maxdim [, thresh=
        thresh])
        # Returns H0 through H_maxdim diagrams
```

- `maxdim` (*int*) — highest homology dimension to compute (0 = connected components, 1 = loops). Default: 1.
- `thresh` (*float* / *None*) — maximum filtration value; edges with weight above this are ignored. *None* disables thresholding.

The returned `PersistenceDiagram` stores death values of $+\infty$ for features that never die (standard `riper` behaviour).

5.2 Diagram Calibration

src/cloudforger/tda/calibration.py

```
def calibrate(
    diagrams: list[PersistenceDiagram],
    percentiles: tuple[float, ...] = (95.0, 99.0),
) -> dict[int, dict[str, dict[float, float]]]:
    ...
```

Pools all finite birth and persistence values across all diagrams and all samples, then computes the requested percentiles. Return value structure:

```
{
  0: { "birth":      {95.0: 0.12, 99.0: 0.18},
        "persistence": {95.0: 0.09, 99.0: 0.14} },
  1: { "birth":      {95.0: 0.20, 99.0: 0.31},
        "persistence": {95.0: 0.15, 99.0: 0.22} }
}
```

`calibrate_report(diagrams, percentiles)` returns a formatted string with the same statistics and suggested `birth_range` / `pers_range` values ready to paste into imager construction.

5.3 Vectorization

5.3.1 PersistenceImager

`src/cloudforger/tda/vectorizer/persistence_image.py`

Converts a single homology dimension of a `PersistenceDiagram` into a fixed-size 2-D raster in *birth-persistence* coordinates.

```
class PersistenceImager:
    def __init__(
        self,
        birth_range: tuple[float, float], # (b_min, b_max)
        pers_range: tuple[float, float], # (p_min, p_max)
        resolution: int = 64, # pixels per axis
        sigma: float = 0.1, # Gaussian std-dev
        weight_fn: Callable = linear_weight,
    ): ...

    def transform(self, diagram: PersistenceDiagram,
                  dim: int) -> np.ndarray:
        # Returns (resolution, resolution) image, vertically flipped
        # so persistence increases upward
        ...

    @property
    def params(self) -> dict: ...
```

Algorithm:

1. Extract finite pairs (b, d) and convert to (b, p) where $p = d - b$.
2. Weight each pair: $w_i = \text{weight_fn}(p_i)$ (default: $w_i = p_i$, the standard linear weight that gives zero weight to diagonal noise).
3. For each pair, add a 2-D Gaussian centred at (b_i, p_i) with standard deviation σ , scaled by w_i , to the image.
4. Flip vertically (row 0 = highest persistence).

Important: bounds are fixed at construction time and *never* inferred from individual diagrams. This ensures that pixels carry consistent spatial meaning across the entire dataset. Pairs outside the bounds still contribute their Gaussian tails to in-range pixels — nothing is silently discarded.

5.3.2 MultiChannelImager

`src/cloudforger/tda/vectorizer/multi_channel.py`

```
class MultiChannelImager:
    def __init__(self, imagers: dict[int, PersistenceImager]):
        # e.g. {0: imager_h0, 1: imager_h1}
        ...

    def transform(self, diagram: PersistenceDiagram
                  ) -> dict[int, np.ndarray]:
```

```
# Returns {dim: (resolution, resolution) array}
...
```

Each homology dimension uses its own independently calibrated imager, allowing H0 and H1 to live in different coordinate systems. The output dictionary is consumed by `PersistenceImageDataset` and routed to per-dimension `PIEncoder` instances.

6 Statistics Subsystem

6.1 CloudStatistic (abstract)

src/cloudforger/stats/base.py

```
class CloudStatistic(ABC):
    def __init__(self, n_samples: int, grid_size: int):
        # Pre-computes a uniform grid over self.grid_range
        ...

    def compute(self, cloud: PointCloud) -> np.ndarray:
        # 1. Calls _sample_values(cloud, rng) to get raw samples
        # 2. Sorts them
        # 3. Returns ECDF evaluated at grid_size evenly spaced points
        # using np.searchsorted (values <= grid[i]) / n_samples
        ...
```

The RNG seed is derived from `cloud.seed + 1_000_003` to avoid correlation with the generation seed, while remaining deterministic.

6.2 PairDistanceCDF

src/cloudforger/stats/pair_dist.py

```
class PairDistanceCDF(CloudStatistic):
    # grid_range = (0.0, 1.0)

    def _sample_values(self, cloud, rng) -> np.ndarray:
        # Samples n_samples random pairs (i, j) with i != j
        # Returns distances / region_diameter
        ...
```

Distances are normalised by the diameter of the cloud’s region (the Euclidean diagonal of the Box), not by per-cloud maximum distance. This is a deliberate design choice: the normalization is identical for all clouds from the same domain, so it does not leak cloud-specific spatial extent into the feature.

Requirement: `cloud.region` must be set (non-None) and must be an instance of `Box`. Clouds generated via `PointProcess.sample()` always set `region` automatically.

7 Neural Network Subsystem

7.1 Datasets

src/cloudforger/nn/data.py

Three PyTorch Dataset subclasses serve as the bridge from `cloudforgery` data objects to the training loop.

7.1.1 PointCloudDataset

```
class PointCloudDataset(Dataset):
    def __init__(self, clouds: list[PointCloud],
                 labels: np.ndarray,
                 n_points: int | None = None,
                 dtype: torch.dtype = torch.long): ...

    def __getitem__(self, idx) -> tuple[torch.Tensor, torch.Tensor]:
        # Returns (float32 tensor of shape (n_points, d), label)
        # If n_points is set, random subsamples per item (with
        # replacement
        # if the cloud has fewer points than n_points)
```

7.1.2 PersistenceImageDataset

```
class PersistenceImageDataset(Dataset):
    def __init__(self, images: list[dict], # list of {dim: ndarray}
                 labels: np.ndarray,
                 dtype: torch.dtype = torch.long): ...

    def __getitem__(self, idx) -> tuple[dict[str, torch.Tensor],
                                       torch.Tensor]:
        # Returns ({"h0": tensor(1,H,W), "h1": tensor(1,H,W), ...},
        #          label)
        # Each image is unsqueezed to add a channel dimension
```

7.1.3 CorrelationFeatureDataset

```
class CorrelationFeatureDataset(Dataset):
    def __init__(self,
                 features: list[CorrelationFeatures | dict],
                 labels: np.ndarray,
                 statistic_names: list[str] | None = None,
                 dtype: torch.dtype = torch.long): ...

    def __getitem__(self, idx) -> tuple[torch.Tensor, torch.Tensor]:
        # Returns (1-D float32 vector, label)

    @property
    def input_dim(self) -> int:
        # Length of concatenated feature vector; pass to StatsEncoder
        ...
```

Accepts both `CorrelationFeatures` objects and raw dicts (coerced automatically). `statistic_names` enables ablation: pass a subset to concatenate only those statistics.

7.2 Encoders

All encoders extend `Encoder`: `src/cloudforgery/nn/encoders/base.py`

```

class Encoder(nn.Module):
    def __init__(self, embedding_dim: int): ...

    @abstractmethod
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Input and output shapes depend on subclass
        ...

    @property
    @abstractmethod
    def input_modality(self) -> str:
        # "point_cloud" / "persistence_image" / "correlation_features"
        "
        ...

```

7.2.1 PointNetEncoder

src/cloudforger/nn/encoders/point_cloud.py

```

class PointNetEncoder(Encoder):
    def __init__(self, input_dim: int,
                  embedding_dim: int,
                  hidden_dims: tuple[int,...] = (64, 128, 256)): ...

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (batch, n_points, input_dim)
        # 1. Apply shared MLP point-wise -> (batch, n_points, hidden
        #    [-1])
        # 2. Max-pool over n_points -> (batch, hidden[-1])
        # 3. Linear head -> (batch, embedding_dim)

```

Implements the permutation-invariant PointNet backbone: each point is processed independently by the same MLP, then the per-cloud representation is the element-wise maximum over all points.

7.2.2 PIEncoder

src/cloudforger/nn/encoders/persistence_image.py

```

class PIEncoder(Encoder):
    def __init__(self, in_channels: int = 2,
                  embedding_dim: int = 128): ...

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (batch, in_channels, H, W)
        # Conv2d(in_ch, 32, 3) -> ReLU -> MaxPool(2)
        # Conv2d(32, 64, 3) -> ReLU -> MaxPool(2)
        # Conv2d(64, 128, 3) -> ReLU -> AdaptiveAvgPool(1)
        # Flatten -> Linear(128, embedding_dim)
        # Output: (batch, embedding_dim)

```

Note on in_channels: PersistenceImageDataset returns each dimension with one channel (shape (1, H, W)). When used in a MultiModalModel with one encoder per dimension, set in_channels=1. The default value 2 targets a configuration where H0 and H1 images are stacked into a two-channel tensor before a single encoder.

7.2.3 StatsEncoder

src/cloudforger/nn/encoders/stats.py

```
class StatsEncoder(Encoder):
    def __init__(self, input_dim: int,
                  embedding_dim: int = 128,
                  hidden_dims: tuple[int,...] = (128, 128)): ...

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (batch, input_dim)
        # MLP with hidden_dims -> Linear(hidden[-1], embedding_dim)
        # Output: (batch, embedding_dim)
```

7.3 Task Heads

7.3.1 ClassificationHead

src/cloudforger/nn/heads/classifier.py

```
class ClassificationHead(nn.Module):
    def __init__(self, embedding_dim: int,
                  n_classes: int,
                  hidden_dims: tuple[int,...] = (128,),
                  dropout: float = 0.1): ...

    def forward(self, embedding: torch.Tensor) -> torch.Tensor:
        # MLP: Linear->ReLU->Dropout (repeated), then Linear(prev,
        # n_classes)
        # Output: (batch, n_classes) raw logits
```

7.3.2 ParameterEstimator

src/cloudforger/nn/heads/paramest.py

```
class ParameterEstimator(nn.Module):
    def __init__(self, embedding_dim: int,
                  n_params: int,
                  hidden_dims: tuple[int,...] = (64,),
                  dropout: float = 0.1): ...

    def forward(self, embedding: torch.Tensor) -> torch.Tensor:
        # Same MLP structure as ClassificationHead
        # Output: (batch, n_params) real-valued regression targets
```

No activation is applied to the output layer — the caller is responsible for any post-processing (e.g. softplus for positivity constraints).

7.4 Models

7.4.1 SingleModalModel

src/cloudforger/nn/models/single_modal.py

```
class SingleModalModel(nn.Module):
    def __init__(self, encoder: Encoder, head: nn.Module): ...
```

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
    return self.head(self.encoder(x))
```

The modality property delegates to `encoder.input_modality`.

7.4.2 MultiModalModel

src/cloudforger/nn/models/multi_modal.py

```
class MultiModalModel(nn.Module):
    def __init__(self, encoders: dict[str, Encoder],
                 head: nn.Module): ...

    @property
    def total_embedding_dim(self) -> int:
        return sum(enc.embedding_dim for enc in self.encoders.values
                    ())

    def forward(self, inputs: dict[str, torch.Tensor]
                ) -> torch.Tensor:
        # For each name in self.encoders:
        #     embeddings.append(encoder(inputs[name]))
        # fused = torch.cat(embeddings, dim=-1)
        # return self.head(fused)
```

Encoders are stored in an `nn.ModuleDict`, so PyTorch tracks their parameters correctly. Missing input keys raise `KeyError`.

Figure 2 shows the architecture options.

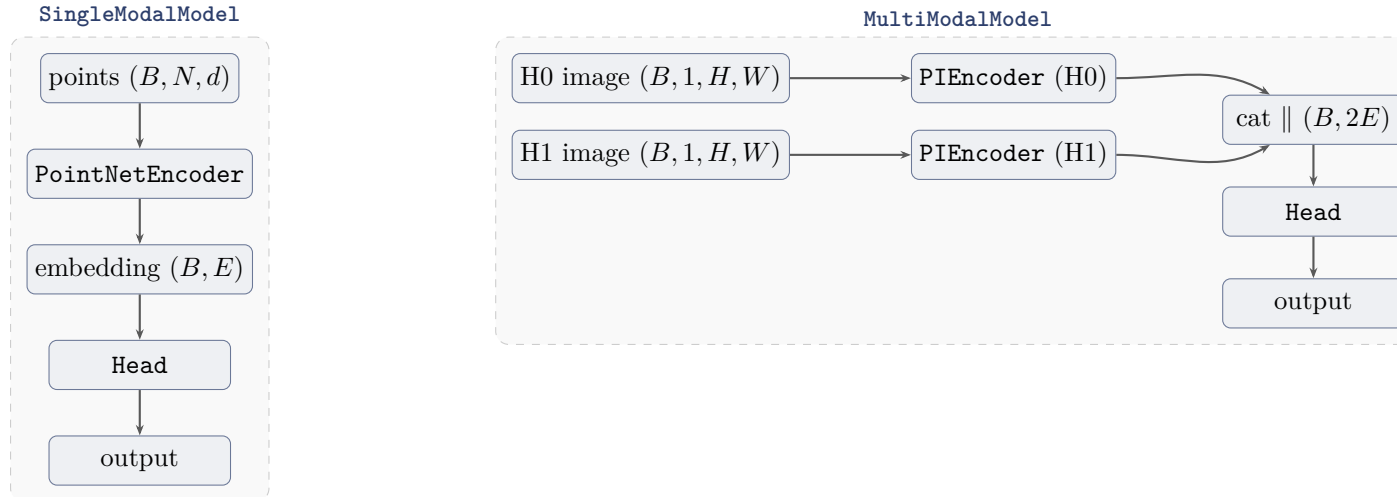


Figure 2: Left: `SingleModalModel` with `PointNetEncoder`. Right: `MultiModalModel` fusing H0 and H1 persistence-image encoders via concatenation before the shared head.

7.5 Training Utilities

7.5.1 train_val_test_split()

src/cloudforger/nn/splits.py

```
def train_val_test_split(
    dataset: Dataset,
    fractions: tuple[float, float, float] = (0.7, 0.15, 0.15),
    seed: int = 0,
) -> tuple[Subset, Subset, Subset]:
    # Splits randomly; fractions must sum to 1.
    # Remainder points (from rounding) go to the test split.
```

7.5.2 train_one_epoch() and evaluate()

src/cloudforger/nn/train.py

```
def train_one_epoch(
    model, loader: DataLoader,
    optimizer: torch.optim.Optimizer,
    loss_fn: nn.Module,
    device: torch.device,
) -> tuple[float, float]:
    # Sets model.train(), iterates loader.
    # Returns (avg_loss, accuracy) for the epoch.
    # Handles dict inputs for MultiModalModel via _to_device().

@torch.no_grad()
def evaluate(
    model, loader: DataLoader,
    loss_fn: nn.Module,
    device: torch.device,
) -> tuple[float, float]:
    # Sets model.eval(), no gradients.
    # Returns (avg_loss, accuracy).
```

Both functions handle both tensor and dict inputs transparently via the internal `_to_device(inputs, device)` helper. Accuracy is computed only for 1-D integer labels (`argmax(logits) == labels`); for regression targets the accuracy value is not meaningful.

7.6 standardize_size()

src/cloudforger/core/utils.py

```
def standardize_size(cloud: PointCloud, n: int,
                    seed: int) -> PointCloud:
    # Raises ValueError if cloud.n_points < n.
    # Returns cloud unchanged if n_points == n.
    # Otherwise samples n points without replacement.
```

Used by the offline processing scripts to normalise cloud sizes before saving datasets; not called by `PointCloudDataset` (which subsamples on-the-fly per batch).

8 Public API Quick Reference

Callable	Description
<code>Box(low, high)</code>	Create an axis-aligned rectangular domain.
<code>PoissonProcess(intensity)</code>	Homogeneous Poisson process; <code>intensity=None</code> gives the conditional (fixed-count) variant.
<code>ThomasProcess(parent_intensity, mean_offspring, cluster_scale)</code>	Clustered Neyman–Scott process with Gaussian offspring kernel.
<code>MaternHardCoreProcess(parent_intensity, intensity, hardcore_radius)</code>	Intensity hard-core process (Matérn Type II).
<code>process.sample(n, region, seed)</code>	Public entry point for all processes. Returns a <code>PointCloud</code> with full provenance metadata.
<code>standardize_size(cloud, n, seed)</code>	Deterministically subsample a cloud to exactly <code>n</code> points.
<code>RipsFiltration(maxdim, thresh)</code>	Configure a Rips filtration.
<code>filtration.compute(cloud)</code>	Compute persistent homology; returns <code>PersistenceDiagram</code> .
<code>PersistenceDiagram.finite_pairs(dim)</code>	Extract birth–death pairs with finite death time.
<code>calibrate(diagrams, percentiles)</code>	Pool statistics over a collection of diagrams; return percentile bounds for birth and persistence per dimension.
<code>calibrate_report(diagrams, percentiles)</code>	Human-readable version of <code>calibrate()</code> .
<code>PersistenceImager(birth_range, pers_range, resolution, sigma, weight_fn)</code>	Construct a fixed-resolution rasterizer for one homology dimension.
<code>imager.transform(diagram, dim)</code>	Rasterize dimension <code>dim</code> of a diagram into a <code>(resolution, resolution)</code> float array.
<code>MultiChannelImager(imagers)</code>	Bundle per-dimension imagers; produces <code>dict[int, ndarray]</code> .
<code>multi.transform(diagram)</code>	Apply all configured imagers to a diagram.
<code>PairDistanceCDF(n_samples, grid_size)</code>	Pair-distance empirical CDF statistic.
<code>statistic.compute(cloud)</code>	Compute the ECDF vector; returns 1-D float array of length <code>grid_size</code> .
<code>PointCloudDataset(clouds, labels, n_points)</code>	PyTorch <code>Dataset</code> over raw point clouds.
<code>PersistenceImageDataset(images, labels)</code>	PyTorch <code>Dataset</code> over persistence-image dicts.
<code>CorrelationFeatureDataset(features, labels, statistic_names)</code>	PyTorch <code>Dataset</code> over correlation-feature vectors.
<code>PointNetEncoder(input_dim, embedding_dim, hidden_dims)</code>	PointNet backbone for raw point clouds.
<code>PIEncoder(in_channels, embedding_dim)</code>	CNN encoder for persistence images.

Callable	Description
<code>StatsEncoder(input_dim, embedding_dim, hidden_dims)</code>	MLP encoder for flat feature vectors.
<code>ClassificationHead(embedding_dim, n_classes, hidden_dims, dropout)</code>	MLP head producing class logits.
<code>ParameterEstimator(embedding_dim, n_params, hidden_dims, dropout)</code>	MLP head producing real-valued regression targets.
<code>SingleModalModel(encoder, head)</code>	Composes one encoder with one head.
<code>MultiModalModel(encoders, head)</code>	Late-fusion model: concatenates embeddings from multiple encoders.
<code>train_val_test_split(dataset, fractions, seed)</code>	Reproducible random split into train/val/test subsets.
<code>train_one_epoch(model, loader, optimizer, loss_fn, device)</code>	One full pass over a <code>DataLoader</code> ; returns (loss, accuracy).
<code>evaluate(model, loader, loss_fn, device)</code>	Evaluation pass (no gradients); returns (loss, accuracy).

9 Usage Walkthrough

This section shows the minimal code needed to go from nothing to a trained single-modal classifier, using the TDA pathway. The example is self-contained and can be run directly after installing the package dependencies.

9.1 Step 1: Generate Clouds

```
import numpy as np
from cloudforger.core.region import Box
from cloudforger.processes.poisson import PoissonProcess
from cloudforger.processes.thomas import ThomasProcess
from cloudforger.processes.matern import MaternHardCoreProcess

region = Box(low=np.zeros(2), high=np.ones(2))

# Three processes, 100 clouds each
processes = [
    (PoissonProcess(), label := 0),
    (ThomasProcess(parent_intensity=20.0,
                    mean_offspring=10.0,
                    cluster_scale=0.05), label := 1),
    (MaternHardCoreProcess(parent_intensity=2000.0,
                           hardcore_radius=0.02), label := 2),
]

clouds, labels = [], []
for proc, lbl in [
    (PoissonProcess(), 0),
    (ThomasProcess(20.0, 10.0, 0.05), 1),
```

```

        (MaternHardCoreProcess(2000.0, 0.02), 2),
]:
    for seed in range(100):
        cloud = proc.sample(n=500, region=region, seed=seed)
        clouds.append(cloud)
        labels.append(lbl)

labels = np.array(labels)

```

9.2 Step 2: Compute Persistence Diagrams

```

from cloudforger.tda.filtration.rips import RipsFiltration

filt = RipsFiltration(maxdim=1)
diagrams = [filt.compute(c) for c in clouds]

```

9.3 Step 3: Calibrate and Vectorize

```

from cloudforger.tda.calibration import calibrate
from cloudforger.tda.vectorizer.persistence_image import
    PersistenceImager
from cloudforger.tda.vectorizer.multi_channel import
    MultiChannelImager

stats = calibrate(diagrams, percentiles=(99.0,))

imagers = {}
for dim in [0, 1]:
    b_hi = stats[dim]["birth"][99.0]
    p_hi = stats[dim]["persistence"][99.0]
    imagers[dim] = PersistenceImager(
        birth_range=(0.0, b_hi),
        pers_range=(0.0, p_hi),
        resolution=64,
        sigma=0.01,
    )

mc = MultiChannelImager(imagers)
images = [mc.transform(d) for d in diagrams]
# images[i] == {0: ndarray(64,64), 1: ndarray(64,64)}

```

9.4 Step 4: Build Datasets and Splits

```

import torch
from torch.utils.data import DataLoader
from cloudforger.nn.data import PersistenceImageDataset
from cloudforger.nn.splits import train_val_test_split

dataset = PersistenceImageDataset(images, labels)
train_ds, val_ds, test_ds = train_val_test_split(
    dataset, fractions=(0.7, 0.15, 0.15), seed=0
)

```

```

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_ds,   batch_size=64)
test_loader  = DataLoader(test_ds,  batch_size=64)

```

9.5 Step 5: Assemble and Train a Model

```

import torch.nn as nn
from cloudforgerr.nn.encoders.persistence_image import PLEncoder
from cloudforgerr.nn.models.multi_modal import MultiModalModel
from cloudforgerr.nn.heads.classifier import ClassificationHead
from cloudforgerr.nn.train import train_one_epoch, evaluate

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# One encoder per homology dimension
encoders = {
    "h0": PLEncoder(in_channels=1, embedding_dim=128),
    "h1": PLEncoder(in_channels=1, embedding_dim=128),
}
head = ClassificationHead(
    embedding_dim=256, # 128 + 128
    n_classes=3,
    hidden_dims=(128,),
    dropout=0.1,
)
model = MultiModalModel(encoders=encoders, head=head).to(device)

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

best_val = float("inf")
for epoch in range(50):
    tr_loss, tr_acc = train_one_epoch(
        model, train_loader, optimizer, loss_fn, device)
    vl_loss, vl_acc = evaluate(
        model, val_loader, loss_fn, device)
    if vl_loss < best_val:
        best_val = vl_loss
        torch.save(model.state_dict(), "best_model.pt")
    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1:3d}: "
              f"train acc={tr_acc:.3f} val acc={vl_acc:.3f}")

# Reload best and evaluate on test set
model.load_state_dict(torch.load("best_model.pt"))
te_loss, te_acc = evaluate(model, test_loader, loss_fn, device)
print(f"Test accuracy: {te_acc:.4f}")

```

10 Known Ambiguities and Implementation Notes

1. **PLEncoder default in_channels=2.** The dataset always produces 1-channel images (`unsqueeze(0)`). The default of 2 targets a use case where H0 and H1 images are stacked before a *single* encoder, but this is not wired up anywhere in the current runners. Use

`in_channels=1` when pairing with `PersistenceImageDataset`.

2. **PointProcess.sample() and the n parameter.** For `ThomasProcess` and `MaternHardCoreProcess`, `n` is passed to `_sample_points` but ignored — the output count is always random. Callers should not rely on exact counts from these processes; use `standardize_size()` or calibrate the parent intensity to achieve the desired expected count.
3. **Matérn thinning loop.** The thinning in `MaternHardCoreProcess._sample_points` uses a Python-level loop over all parents. For large parent intensities (many thousands of points) this is the performance bottleneck.
4. **Persistence imager clipping.** Gaussian kernels are evaluated for all pairs, including those outside `birth_range / pers_range`. Out-of-range pairs still contribute tails to in-range pixels. This matches the spirit of the Adams et al. persistence image definition but can cause mild boundary effects near the image edges.
5. **Module naming discrepancy.** Some file-level comments and the `train.py` module header reference the name `pointforge` (the older package name) rather than `cloudforgery`. The public import paths use `cloudforgery` throughout; the stale comments are cosmetic.
6. **Parameter estimation accuracy metric.** `train_one_epoch` and `evaluate` compute accuracy via `argmax(logits) == labels`, which is only meaningful for integer class labels. When using `ParameterEstimator`, the returned accuracy is meaningless and should be ignored; log the raw loss instead.